

Dokumentasi Teknis — KidzPlayful

Dokumen ini menjelaskan **seluruh alur** aplikasi KidzPlayful dari nol sampai deploy: arsitektur, tiap berkas & perannya, parameter penting, skema database, serta cara deploy ke **Vercel** (frontend) dan **Supabase** (backend). Tujuannya agar Anda paham FE & BE secara menyeluruh.

- **Aplikasi:** web app kelas bermain digital anak 0–4 tahun.
 - **Repo:** github.com/khabibrizal/kidzplayful · **Live:** <https://kidzplayful-fe2a.vercel.app>
 - **Stack:** Next.js 16 (App Router, TypeScript) + Supabase (Postgres + Auth + Storage). Tanpa server backend terpisah — "backend" = Supabase + Server Actions/Server Components Next.js.
-

1. Arsitektur Singkat (FE & BE)

```
Browser (HP/Tablet/Laptop)
  | HTML/JS dari Next.js
  ▼
Next.js (di Vercel)                                ← "Frontend + lapisan server"
├─ Server Components : render halaman di server, baca data (aman)
├─ Client Components : interaktif di browser ('use client')
├─ Server Actions    : fungsi tulis di server ('use server')
├─ proxy.ts (middleware): segarkan sesi login tiap request
├─ panggil via @supabase/ssr (pakai cookie sesi)
  ▼
Supabase (cloud)                                    ← "Backend"
├─ Auth      : akun & sesi (email+password)
├─ Postgres  : tabel data + RLS (keamanan baris) + trigger/function
├─ Storage   : file (gambar game, worksheet PDF) di bucket 'aset'
```

Konsep kunci: tidak ada "server Express/Node" terpisah. Logika server berjalan **di dalam Next.js** (Server Components untuk baca, Server Actions untuk tulis), dan **keamanan data utama ada di Supabase RLS** (Row Level Security) — aturan di database yang menentukan baris mana boleh dibaca/ditulis oleh siapa.

2. Tech Stack & Alasan

Teknologi	Untuk apa	Kenapa
Next.js 16 (App Router)	Framework FE + server	Satu kerangka untuk UI + server logic; deploy mudah ke Vercel
TypeScript	Bahasa	Tipe data → lebih sedikit bug
Supabase	Auth + Database + Storage	Backend siap pakai (Postgres + RLS), gratis untuk mulai
@supabase/ssr	Hubungkan Next.js ↔ Supabase	Mengelola sesi login lewat cookie (server & browser)
Vitest	Unit test (logika murni)	Cepat, untuk fungsi di <code>lib/domain</code>
Playwright	End-to-end test (alur nyata di browser)	Uji daftar→login→main, dll
Vercel	Hosting frontend	Auto-deploy tiap <code>git push</code>
CSS (global + module)	Styling	Ringan; tema pastel + maskot Pewi

3. Menjalankan di Lokal

Prasyarat: **Node.js 20+**, akun **Supabase**, file `.env.local`.

```
cd d:\kidzplayful
npm install           # pasang dependency
# buat .env.local (lihat bagian 4)
npm run dev           # jalan di http://localhost:3000
npm test              # unit test (Vitest)
npm run e2e           # end-to-end test (Playwright)
npm run build         # build produksi (cek error)
npm run lint          # ESLint
```

Script (`package.json`): - `dev` → `next dev` (mode pengembangan, hot reload) - `build` → `next build` (kompilasi produksi) - `start` → `next start` (jalankan hasil build) - `test` / `test:watch` → Vitest - `e2e` → Playwright

4. Variabel Lingkungan (`.env.local`)

File `.env.local` (TIDAK ikut ke Git — rahasia). Di Vercel diisi di **Settings** → **Environment Variables**.

Variabel	Untuk apa	Contoh
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Alamat proyek Supabase	<code>https://xxxx.supabase.co</code>
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Kunci publishable (aman dipakai di browser)	<code>sb_publishable_...</code>

Penting: awalan `NEXT_PUBLIC_` membuat variabel **disuntik ke bundel browser saat build** — jadi harus ada **sebelum build** (di Vercel & lokal). Kunci publishable aman karena keamanan sebenarnya dijaga oleh **RLS** di Supabase, bukan oleh kerahasiaan kunci ini. **JANGAN** memakai *secret key* di sini.

`.env.local.example` = template (boleh ikut Git). `.gitignore` mengecualikan `.env*` tapi mengizinkan `.env.local.example`.

5. Konsep Next.js yang Dipakai (wajib paham)

- **Server Component** (default, file `page.tsx` / `layout.tsx` tanpa `'use client'`): dijalankan **di server**, boleh `await` ambil data dari Supabase, aman (kode tak terkirim ke browser). Dipakai untuk halaman yang membaca data.
- **Client Component** (`'use client'` di baris atas): dijalankan **di browser**, punya `useState` / `onClick` / interaktivitas. Dipakai untuk form, tombol, game.
- **Server Action** (`'use server'`): fungsi yang berjalan **di server** tapi bisa dipanggil dari Client Component — untuk **menulis** data (insert/update/delete) dengan aman. Mis. `buatPostingan`, `aktifkanLangganan`.
- **params & searchParams**: di Next 16 berupa **Promise** → harus `await`. `params` = bagian URL dinamis (mis. `[anakId]`), `searchParams` = query (`?paket=...`).
- **redirect()** (`next/navigation`): pindah halaman dari server (mis. guard menendang user tak berhak).
- **revalidatePath('/...')**: setelah menulis data, memberi tahu Next agar halaman itu diambil ulang (data segar).
- **proxy.ts** (dulu `middleware.ts`): berjalan tiap request untuk **menyegarkan cookie sesi** Supabase.

Rute = struktur folder di `src/app/`. Folder `[x]` = parameter dinamis. `page.tsx` = halaman, `layout.tsx` = kerangka pembungkus.

6. Supabase (Backend) — Konsep

- **Auth**: menyimpan user (email+password) & sesi. Saat daftar, dipicu trigger membuat baris `profiles` + `langganan` trial.
- **Postgres + RLS**: setiap tabel punya **policy** (aturan) yang menentukan siapa boleh `select/insert/update/delete` baris mana. Contoh: "anak hanya bisa dibaca oleh ortu pemiliknya". RLS = lapisan keamanan utama.
- **Fungsi** `public.is_admin()`: mengembalikan `true` bila user saat ini admin (cek `profiles.is_admin`). Dipakai di banyak policy admin.
- **Storage**: bucket `aset` (publik baca, tulis khusus admin) untuk gambar game & worksheet PDF.
- **Dua client Supabase di kode:**

- `src/lib/supabase/client.ts` → **browser** (`createBrowserClient`), dipakai di Client Component.
- `src/lib/supabase/server.ts` → **server** (`createServerClient` , baca cookie), dipakai di Server Component & Server Action.

7. Skema Database (per tabel)

Diterapkan lewat **migrasi** `supabase/migrations/0001..0014` (file SQL, dijalankan di Supabase SQL Editor). Ringkas:

profiles (0001, +kolom di 0004/0010) — profil orang tua (1-1 dgn `auth.users`)

Kolom	Tipe	Arti
<code>id</code>	uuid (PK, =auth.users.id)	identitas user
<code>email</code>	text	email login
<code>pin_ortu</code>	text	PIN 4 angka Gerbang Orang Tua
<code>is_admin</code>	bool	penanda admin (default false)
<code>nama_tampilan</code>	text	nama publik di komunitas
<code>terakhir_aktif</code> , <code>created_at</code>	timestampz	waktu

RLS: user baca/ubah **profil sendiri**; admin boleh baca semua. **Trigger** `cegah_self_admin` (0012): user biasa **tak bisa** mengubah `is_admin` sendiri (anti self-promote) — hanya via SQL Editor.

anak (0001) — profil anak

`id`, `ortu_id`(FK profiles), `nama`, `tanggal_lahir`, `mode_default`('ortu'|'anak'), `batas_menit`, `koin`, `created_at` . RLS: hanya milik ortu (`auth.uid()`=`ortu_id`).

langganan (0001, +nominal di 0004) — langganan per ortu (1-1)

`id`, `ortu_id`, `status`('trial'|'aktif'|'menunggu'|'tenggang'|'kadaluarsa'), `nominal`, `trial_mulai`, `trial_selesai`, `aktif_sampai`, `dibayar_via`, `diaktifkan_oleh`, `updated_at` . RLS: baca milik sendiri; admin baca + update (aktivasi). **Trigger** `handle_new_user` (0001): saat user daftar → buat `profiles` + `langganan` (trial 14 hari) otomatis.

tema (0002) — tema mingguan (mis. Hewan)

`id`, `nama`, `sampul`(emoji), `status`('draf'|'disetujui'), `is_minggu_ini`(bool), `jadwal_tayang` . RLS: baca yang disetujui (authenticated); admin kelola.

paket_aset (0002, +usia 0005, +sumber/status) — konten 1 game per tema

`id`, `tema_id`, `mesin`('tekan-sesuai'|'seret-wadah'|'cari-pasangan'|...), `judul`, `area_skill`, `usia_min`, `usia_max`, `sumber`('ai'|'manual'), `status`, `butir`(jsonb), `urutan` . `butir` = JSON soal/aset game (inti

"ganti tema tanpa koding").

hasil_main (0002) — log skor tiap sesi main

id, anak_id, tema_id, mesin, area_skill, jumlah_coba, selesai, durasi_detik, bintang, tanggal . RLS: milik anak dari ortu yg login.

video (0003, +kategori 0005) — video Pojok Video

id, tema_id(nullable), judul, youtube_id, durasi_detik, urutan, link_ok, kategori('baby'|'toddler'), status . RLS: baca yang disetujui & link_ok; admin kelola.

panduan (0008, +field 0009/0013) — LEGACY (digantikan kelas_bermain)

Dulu materi kelas bermain per tema. Kini tak dipakai aktif.

kelas_bermain (0014) — materi kelas bermain mandiri (lepas tema)

Kolom	Arti
id	identitas
judul	judul kelas bermain
aktivitas	deskripsi aktivitas
bahan	bahan/alat
cara_membuat	cara membuat (teks bebas)
langkah	jsonb array langkah
link_ide	URL referensi ide
worksheet_url	URL PDF worksheet (di Storage)
status	'aktif' 'nonaktif'
RLS: baca yang aktif (authenticated) / semua (admin); admin kelola. Ditampilkan di Mode Anak & Mode Ortu.	

postingan / komentar / suka (0010) — komunitas

- postingan : id, ortu_id, nama(snapshot nama tampilan), tema_id(nullable), teks, status('tampil'|'disembunyikan'), created_at .
- komentar : id, postingan_id, ortu_id, nama, teks, status, created_at .
- suka : (postingan_id, ortu_id) PK gabungan (1 like/ortu). RLS: baca yang tampil ; tulis milik sendiri; **admin** boleh sembunyikan/hapus.

laporan (0011) — laporan moderasi komunitas

`id, postingan_id(nullable), komentar_id(nullable), pelapor, alasan, created_at` . RLS: user insert; admin baca/hapus.

Daftar migrasi (urutan jalankan): 0001 init → 0002 konten → 0003 video+tema2 → 0004 admin → 0005 video kategori → 0006 admin bisnis (RLS) → 0007 storage → 0008 panduan → 0009 kelas bermain (field) → 0010 komunitas → 0011 moderasi → 0012 cegah self-admin → 0013 field kelas → 0014 kelas_bermain mandiri.

8. Struktur Folder & Peran Tiap Berkas

`src/lib/supabase/` — koneksi Supabase

- `client.ts` — Supabase client **browser** (Client Components).
- `server.ts` — Supabase client **server** (Server Components/Actions; baca cookie sesi).

`src/lib/domain/` — logika murni (tanpa DB, mudah diuji)

Berkas	Fungsi & parameter
<code>trial.ts</code>	<code>computeTrialEnd(mulai)</code> (+14 hari); <code>statusLangganan({trialMulai,aktifSampai},sekarang)</code> → 'aktif'/'trial'/'tenggang'/'kadaluarsa'; <code>bolehAkses(status)</code>
<code>anak.ts</code>	<code>umurTahun(tglLahir, sekarang)</code> ; <code>modeDefault(umur)</code> → <2 'ortu', ≥2 'anak'
<code>usia.ts</code>	<code>cocokUsia(umur,min,max)</code> ; <code>kategoriUsia(umur)</code> → <2 'baby', ≥2 'toddler'
<code>skor.ts</code>	<code>hitungBintang(benar,total)</code> → 1–3 bintang
<code>waktu.ts</code>	<code>sisaDetik(terpakai,batasMenit)</code> ; <code>waktuHabis(...)</code> ; <code>kunciHari(anakId,tgl)</code> (kunci localStorage harian)
<code>laporan.ts</code>	<code>ringkasanLangganan(rows,sekarang)</code> → hitung aktif/trial/dll + MRR
<code>laporan-anak.ts</code>	<code>laporanAnak(rows)</code> → total sesi/bintang/menit + per area skill
<code>__tests__/*</code>	Unit test Vitest tiap fungsi di atas

`src/lib/game/` — tipe & util game

- `tipe.ts` — semua **interface** TypeScript (Paket, Video, KelasBermain, Panduan, dll). Sumber tipe data game.
- `butir.ts` — `butirDariForm` , `validasiButir` (validasi isi game per mesin).
- `aset.ts` — `isUrlAset(v)` (cek nilai = URL gambar atau emoji).

src/lib/data/ — lapisan data (baca = fungsi; tulis = Server Action 'use server')

Berkas	Isi
anak.ts	getAnakTerjamin(anakId) — ambil anak + guard (login & langganan aktif)
tema.ts	getMingguIni() (legacy)
pustaka.ts	getPustaka() — semua tema+paket+video utk Game Edukasi
video.ts	getVideoByKategori('baby' 'toddler')
skor.ts	catatHasil(...) (Action) — simpan hasil_main + tambah koin
kelas-bermain.ts	getKelasAktif() / getKelasSemua()
kelas-bermain-actions.ts	Actions admin: buatKelas/updateKelas/toggleStatusKelas/hapusKelas
admin.ts	getAdminTerjamin() — guard halaman admin (cek is_admin)
admin-konten.ts	Actions admin: tema/paket/video/panduan (CRUD konten)
admin-bisnis.ts	aktifkanLangganan(ortuId,nominal,via)
admin-komunitas.ts	Actions moderasi: sembunyikan/hapus postingan/komentar
ortu-actions.ts	Actions ortu: updateAnak/setBatas/hapusAnak/setPin
komunitas.ts	getFeed() / getPostingan(id)
komunitas-actions.ts	Actions: posting/komentar/suka/lapor/nama tampilan
panduan.ts	(legacy) getModeOrtu/getKelasBermain

Pola penting: fungsi **baca** dipanggil dari Server Component; fungsi **tulis** ditandai 'use server' (Server Action) dan dipanggil dari Client Component. Semua guard "milik sendiri" memakai `.eq('id'/'ortu_id', user.id)` agar aman & tidak error untuk admin (yang bisa baca banyak baris).

src/components/ — komponen UI dipakai ulang

- ui/Pewi.tsx — maskot (SVG). ui/Confetti.tsx — animasi konfeti hadiah.
- game/ — engine & elemen game:
- GameRunner.tsx — pemilih engine sesuai mesin + catat skor + tampilkan Reward.
- ManaYa.tsx (tekan), BeresBeres.tsx (seret), CariPasangan.tsx (cocok) — 3 mesin game.
- Aset.tsx — render gambar (URL) atau emoji.
- Reward.tsx — layar bintang+koin+konfeti. PinGate.tsx — gerbang PIN. VideoPojok.tsx — pemutar video terkunci.
- admin/AsetInput.tsx — input aset game: ketik emoji ATAU upload gambar ke Storage.

Rute	Berkas	Untuk
/	page.tsx	Landing (Pewi + tombol Mulai/Masuk)
/daftar , /login	masing-masing	Auth
/pilih-anak	page.tsx + actions.ts	Beranda ortu: daftar/ tambah anak; tautan ke Kelola/PilihGame/Pengaturan/Komunitas/Admin
/main/[anakId]	page.tsx + MenuAnak.tsx	Mode Anak: Kelas Bermain, Game Edukasi, Pojok Video; PIN; batas waktu; game
/ortu/[anakId]	page.tsx	Mode Ortu 0-2: kelas bermain + video baby
/anak/[anakId]	page.tsx + KelolaAnak.tsx	Kelola profil anak (edit/batas/hapus)
/anak/[anakId]/laporan	page.tsx	Laporan perkembangan anak
/pilih-game/[anakId]	page.tsx + PilihGame.tsx	Rekomendasi game sesuai usia → klik langsung main
/pengaturan	page.tsx + AkunForm/PinForm>NamaForm	PIN, ganti sandi, logout, nama tampilan, bayar langganan
/komunitas	page.tsx + Compose/SukaBtn/LaporBtn	Feed komunitas
/komunitas/[postId]	page.tsx + KomentarForm	Detail + komentar
/admin	layout.tsx (guard) + page.tsx	Dashboard admin
/admin/tema/[id]	page.tsx + PaketForm.tsx	Kelola tema + paket game
/admin/kelas-bermain	page.tsx + KelasAdmin.tsx	CRUD kelas bermain (search/edit/nonaktif/hapus/toast/loading)
/admin/video	page.tsx + VideoForm.tsx	Kelola video per kategori
/admin/langganan	page.tsx + AktifkanForm.tsx	Aktivasi langganan manual
/admin/laporan	page.tsx	Laporan member (ringkasan/MRR/keterlibatan)
/admin/komunitas	page.tsx	Moderasi komunitas
globals.css	—	Design tokens pastel + kelas <code>kp-*</code>
error.tsx , not-found.tsx	—	Halaman error/404 ramah anak
layout.tsx	—	Root: font (Baloo/Quicksand) + metadata
proxy.ts	—	Penyegar sesi (middleware Next)

9. Alur Fitur (FE ↔ BE) — contoh

Daftar → Trial: `/daftar` (Client) panggil `supabase.auth.signUp` → Supabase Auth buat user → **trigger** buat `profiles` + `langganan` trial → diarahkan ke `/pilih-anak` (Server membaca status via `statusLangganan`).

Main game: `/main/[anakId]` (Server) ambil anak (`getAnakTerjamin`) + pustaka + kelas bermain → kirim ke `MenuAnak` (Client). Anak pilih game → `GameRunner` jalankan engine (mis. `ManaYa`) → selesai → panggil Action `catatHasil` (tulis `hasil_main` + koin) → `Reward` + konfeti.

Admin kelas bermain: `/admin/kelas-bermain` (Server, guard `getAdminTerjamin`) ambil `getKelasSemua` → `KelasAdmin` (Client) tampilkan list + search; Tambah/Edit panggil Action `buatKelas/updateKelas` → state list diperbarui + **toast**. Upload PDF → ke Storage `aset` .

Aktivasi langganan: ortu lihat instruksi bayar di `/pengaturan` → transfer/QRIS → konfirmasi WhatsApp → admin di `/admin/langganan` klik **Aktifkan** → Action `aktifkanLangganan` set status `aktif` +1 bulan.

10. Keamanan (ringkas tapi penting)

1. **RLS di setiap tabel** — penentu utama siapa boleh apa (bukan kode FE).
 2. **Guard di kode** — `getAnakTerjamin` , `getAdminTerjamin` , `adminDb()` memeriksa login/admin/langganan sebelum aksi.
 3. `is_admin()` **function + trigger anti self-promote (0012)** — user tak bisa menjadikan diri admin; admin diberikan **hanya** via SQL Editor.
 4. **Query "milik sendiri" difilter** `.eq(..user.id)` — mencegah error & kebocoran untuk akun admin.
 5. **Kunci publishable di browser aman; secret key TIDAK dipakai** di app.
 6. **Privasi anak:** Mode Anak tanpa iklan/tautan keluar; video `youtube-nocookie` ; PIN melindungi keluar.
-

11. Testing

- **Unit (Vitest):** `npm test` — menguji `src/lib/domain/*` & `src/lib/game/*` (logika murni: trial, umur, skor, dll). 30 test.
 - **E2E (Playwright):** `npm run e2e` — alur nyata di browser (daftar→trial→tambah anak, main game, pojok video, dll). Butuh `.env.local` + Supabase aktif.
 - `tools/*.mjs` — skrip uji manual (puppeteer) untuk verifikasi produksi (mis. login admin).
-

12. Deploy — Supabase (Backend)

1. Buat proyek di **supabase.com** (gratis).
2. **SQL Editor** → jalankan migrasi `0001` s/d `0014` **berurutan** (isi tiap file di `supabase/migrations/`).

3. **Authentication** → **Providers** → **Email** → matikan "Confirm email" (dev), atau atur SMTP untuk produksi.
4. **Authentication** → **URL Configuration** → **Site URL** = domain Vercel.
5. **Project Settings** → **API** → salin **Project URL** + **publishable key** → masukkan ke env Vercel (& `.env.local` lokal).
6. **Beri admin** (sekali): `update public.profiles set is_admin=true where email='EMAIL_ANDA';`
7. Storage bucket `aset` otomatis dibuat oleh migrasi 0007.

13. Deploy — Vercel (Frontend)

1. Push repo ke **GitHub** (sudah: `khabibrizal/kidzplayful`).
2. `vercel.com` → **Add New** → **Project** → **Import** repo.
3. Framework auto-terdeteksi **Next.js** (Root `./`).
4. **Environment Variables** (Production+Preview+Development): isi `NEXT_PUBLIC_SUPABASE_URL` & `NEXT_PUBLIC_SUPABASE_ANON_KEY` **sebelum** Deploy.
5. **Deploy**. Selanjutnya **tiap** `git push` **ke** `master` → **auto-deploy**.
6. **Catatan plan Hobby**: auto-deploy dari repo **private** diblokir kecuali commit dari pemilik. Solusi kita: repo dibuat **public**. (Alternatif: verifikasi email GitHub / upgrade Pro.)

Alur rilis sehari-hari:

```
# ubah kode → commit → push → Vercel auto-deploy
git add -A && git commit -m "pesan" && git push origin master
```

Bila skema DB berubah: tulis migrasi baru `00NN_*.sql` lalu jalankan di Supabase SQL Editor (lokal & produksi pakai DB Supabase yang sama).

14. Masalah yang Pernah Terjadi & Solusi (riwayat nyata)

Masalah	Sebab	Solusi
Deploy "Blocked" di Vercel	Hobby + repo private	Repo dijadikan public
Beranda tampil boilerplate Next	<code>page.tsx</code> belum diganti	Buat landing KidzPlayful
Login admin malah ke /pilih-anak	Admin bisa baca semua baris → <code>.single()</code> error	Filter <code>.eq('id', user.id)</code> di query profil/langganan
User bisa jadi admin sendiri	RLS profil mengizinkan ubah <code>is_admin</code>	Trigger <code>cegah_self_admin</code> (0012)
Warning "middleware deprecated"	Next 16	Rename <code>middleware.ts</code> → <code>proxy.ts</code>
Diagram Mermaid error di mockup	render runtime gagal	Render jadi SVG statis

15. Glosarium Parameter Penting

- **butir (paket_aset, jsonb)**: isi game per tema. Bentuk beda per mesin (mis. `{soal: [{tanya, benar, salah[]}]}`). Membuat 1 engine bisa dipakai banyak tema tanpa koding ulang.
- **status (berbagai tabel)**: kontrol tampil/tidak (`disetujui/draf`, `aktif/nonaktif`, `tampil/disembunyikan`).
- **mode_default (anak)**: menentukan masuk Mode Anak (2+) atau Mode Ortu (0-2).
- **kategori (video)**: 'baby' (Mode Ortu) / 'toddler' (Mode Anak) sesuai usia.
- **nominal (langganan)**: harga langganan untuk hitung **MRR** di laporan.
- **batas_menit (anak)**: batas screen-time harian; dipantau via localStorage `kunciHari`.
- **is_minggu_ini (tema)**: tema yang ditonjolkan di Game Edukasi.

Dokumen ini mengikuti kode terkini. Bila ada perubahan besar, perbarui file `docs/DOKUMENTASI-KIDZPLAYFUL.md` lalu regenerasi PDF.